


```

- tgt: 'minion1'
- sls:
  - nginx          # This runs your existing nginx state

# Step 2: Health check for Minion 1

check_web_server_1:
  salt.function: # Function Module: Tells the Master to run a single
command/function.
  - name: cmd.run # The Function: Use the 'cmd.run' module on the minion.
  - tgt: 'minion1'
  - arg:
    - curl -f http://localhost
  - require:
    - salt: update_web_server_1

# Step 3: Only if Minion 1 is healthy, update Minion 2

update_web_server_2:
  salt.state:
    - tgt: 'minion2'
    - sls:
      - nginx
    - require: #"Only start Minion 2 if the check on Minion 1 passed."
    - salt: check_web_server_1

```

Step B: Execute the Orchestration that starts a runner process

Step 1.2 : `sudo salt-run state.orchestrate orch.web_rolling_update`

```

+       <h1>Minion ID: minion-2</h1>
+
+
+       <p><em>Thank you for using nginx.</em></p>
+
+
+
+
+       <div class="master-info primary">
+         <h2>MANAGEMENT STATUS</h2>
+         <p>This Minion is currently managed by the:</p>
+
+         <h3>PRIMARY MASTER</h3>
+
+         <p>Master IP Address: 192.168.64.11</p>
+       </div>
+
+     </body>
+   </html>
-----
ID: nginx_service
Function: service.running
Name: nginx
Result: True
Comment: Service restarted
Started: 15:11:18.167368
Duration: 632.17 ms
Changes:
-----
nginx:
  True

Summary for minion-2
-----
Succeeded: 5 (changed=5)
Failed:    0
-----
Total states run:    5
Total run time: 12.819 s

Summary for salt-master_master
-----
Succeeded: 3 (changed=3)
Failed:    0
-----
Total states run:    3
Total run time: 27.551 s

```

If **minion-1** had failed (e.g., a syntax error in the Nginx config), the Orchestration would have **stopped immediately**. **minion-2** would have stayed online with its old configuration, preventing a total site outage.

Now we will purposely add an error in the nginx config file and run the orchestration command to see how the update on minion1 failed and it does not pass the error to the next minion. Our minion-2 will continue to run on the older version avoiding error

In a manual world (or using a simpler script), an engineer might run a command across all servers at once, breaking the entire website.

Through this we achieve -

- **Zero-Downtime Reliability:** Even though your code was "broken," 50% of your infrastructure (Minion-2) stayed online and healthy.

- **Automated Validation:** The orchestration didn't just "try its best"; it validated the outcome before proceeding.

[illegible]

```

    diff:
    ---
    +++
    @@ -1,5 +1,5 @@
    server {
    -     listen 80;
    +     listen 80xxx;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html index.htm;
}

-----
ID: nginx_default_site_absent
Function: file.absent
Name: /etc/nginx/sites-enabled/default
Result: True
Comment: File /etc/nginx/sites-enabled/default is not present
Started: 15:29:39.083343
Duration: 0.225 ms
Changes:

-----
ID: nginx_index
Function: file.managed
Name: /usr/share/nginx/html/index.html
Result: True
Comment: File /usr/share/nginx/html/index.html is in the correct state
Started: 15:29:39.083613
Duration: 14.085 ms
Changes:

-----
ID: nginx_service
Function: service.running
Name: nginx
Result: False
Comment: Running as unit: run-r767176082e7c4c3e8e404fe565e4d7c7.scope; invocation ID: edc20e44aac643e68d9b44add86b9e28
Job for nginx.service failed because the control process exited with error code.
See "systemctl status nginx.service" and "journalctl -xeu nginx.service" for details.
Started: 15:29:39.134433
Duration: 353.14 ms
Changes:

Summary for minion-1
-----
Succeeded: 4 (changed=1)
Failed: 1
-----
Total states run: 5
Total run time: 402.850 ms

-----
ID: check_web_server_1
Function: salt.function
Name: cmd.run
Result: False
Comment: One or more requisite failed: orch.web_rolling_update.update_web_server_1
Started: 15:29:39.464589
Duration: 0.003 ms
Changes:

-----
ID: update_web_server_2
Function: salt.state
Result: False
Comment: One or more requisite failed: orch.web_rolling_update.check_web_server_1
Started: 15:29:39.464697
Duration: 0.002 ms
Changes:

Summary for salt-master_master
-----
Succeeded: 0 (changed=1)
Failed: 3
-----
Total states run: 3
Total run time: 1.610 s
ubuntu@salt-master: /srv/salt/orch$ █

```

Rollback and Email Notifications

```

/srv/salt/
├── nginx/
│   ├── init.sls      # Your standard Nginx state
│   ├── previous_stable.sls # The rollback state we are creating
│   └── files/
│       ├── default.conf.j2 # The template we are editing (and might break)
│       └── stable.conf     # A hard-coded, known-working config file

```

Enhance the Orchestration logic with an **onfail** trigger. If the Nginx update fails the health check, the Master must automatically send an email alert to the team.

```
ubuntu@salt-master:/srv/salt/orch$ sudo mkdir -p /srv/salt/nginx/files
ubuntu@salt-master:/srv/salt/orch$ sudo mv /srv/salt/nginx/nginx_default.conf /srv/salt/nginx/files/default.conf.j2
```

```
None
server {
    listen 80xxx;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html index.htm;
    location / {
        try_files $uri $uri/ =404;
    }
}
```

```
ubuntu@salt-master:/srv/salt/orch$ sudo vim /srv/salt/nginx/files/stable.conf
```

```
None
server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;
}
```

```
ubuntu@salt-master:/srv/salt/orch$ sudo vim /srv/salt/nginx/init.sls
```

```
None
nginx_packages:
  pkg.installed:
    - pkgs:
      - nginx
      - apache2-utils
      - ca-certificates
# Manage the whole directory, but add the shield here
nginx_config_dir:
  file.recurse: # State Module: Syncs an entire folder from Master to Minion.
    - name: /etc/nginx # Where the files go on the Minion.
    - source: salt://nginx/etc/nginx # Where the files are on the Master.
    - include_empty: True # Keeps empty directories intact.
```

```

- template: jinja # Processes Jinja variables
- dir_mode: '0755' # Standard folder permissions.
- file_mode: '0644' # Standard file permissions.
# THE SHIELD: This stops the recurse from happening if the source files are
broken saves the new config in a temp file if its runs successfully its made
permanent else its aborted
- check_cmd: /usr/sbin/nginx -t -c /etc/nginx/nginx.conf -g "include %s;"
- require:
  - pkg: nginx_packages
# This fixes the '80xxx' issue by ensuring the resolver is valid
#Instead of hardcoding a DNS server like 8.8.8.8, this state dynamically steals the
DNS configuration from the host machine.

nginx_dns_resolver:
  cmd.run:
    - name: (echo "resolver "; cat /etc/resolv.conf |awk '{print $2}' ; echo ";") |
tr '\n' ' ' > /etc/nginx/resolver.conf
    - unless: test -f /etc/nginx/resolver.conf && [ "$(stat -c %Y /etc/resolv.conf)"
-lt "$(stat -c %Y /etc/nginx/resolver.conf)" ]
    - require:
      - file: nginx_config_dir

nginx_service:
  service.running:
    - name: nginx
    - enable: True
    - reload: True
    - watch:
      - file: nginx_config_dir
      - cmd: nginx_dns_resolver

```

ubuntu@salt-master:/srv/salt/orch\$ sudo vim /srv/salt/nginx/previous_stable.sls

None

```

# 1. Kill the file that is actually causing the '80xxx' crash
remove_poison_config:
  file.absent:
    - name: /etc/nginx/conf.d/default.conf
# 2. Ensure the site config is back to stable
rollback_nginx_config:
  file.managed:
    - name: /etc/nginx/sites-available/default
    - source: salt://nginx/files/stable.conf

```

```

- user: root
- group: root
- mode: 644
# 3. Now Nginx WILL start because the poison file is gone
restart_nginx_after_rollback:
  service.running:
    - name: nginx
    - enable: True
    - force_restart: True
    - watch:
      - file: rollback_nginx_config

```

ubuntu@salt-master:/srv/salt/orch\$ sudo vim /srv/salt/orch/web_rolling_update.sls

```

None
update_minion_1:
  salt.state:
    - tgt: 'minion-1'
    - sls:
      - nginx.init
rollback_minion_1:
  salt.state:
    - tgt: 'minion-1'
    - sls:
      - nginx.previous_stable
    - onfail:
      - salt: update_minion_1
alert_admin:
  salt.function:
    - name: smtp.send_msg
    - tgt: 'minion-1'
    - kwarg:
        recipient: 'shambhavi.intern@phonepe.com'
        message: 'Nginx deployment failed on minion-1. Rollback has been initiated.'
        subject: 'SaltStack Alert: Deployment Failed'
        sender: 'shambhavi.intern@phonepe.com' # Add this
        server: 'smtp.gmail.com'
        username: 'shambhavi.intern@phonepe.com' # 'user' becomes 'username'
        password: 'iwgtazsqbzayhhxz'
    - onfail:
      - salt: update_minion_1

```

ubuntu@salt-master:/srv/salt/orch\$ sudo vim /etc/salt/master.d/smtp.conf

None

```
smtp.host: 'smtp.gmail.com'
smtp.port: 587
smtp.user: 'shambhavi.intern@phonepe.com'           # Your actual Gmail address
smtp.password: 'iwgtazsqbzayhhxz'                 # The 16-character App Password (no spaces)
smtp.tls: True
smtp.from: 'shambhavi.intern@phonepe.com'          # Must match your Gmail address
```

ubuntu@salt-master:~\$ sudo systemctl restart salt-master

ubuntu@salt-master:~\$ sudo salt-run smtp.send \
subject='SaltStack Alert System Test' \
message='If you see this, your Salt Master is ready to send alerts!' \
recipient='shambhavi.intern@phonepe.com'

'smtp.send' is not available.

ubuntu@salt-master:~\$ sudo apt-get update

ubuntu@salt-master:~\$ sudo apt-get install python3-pip -y

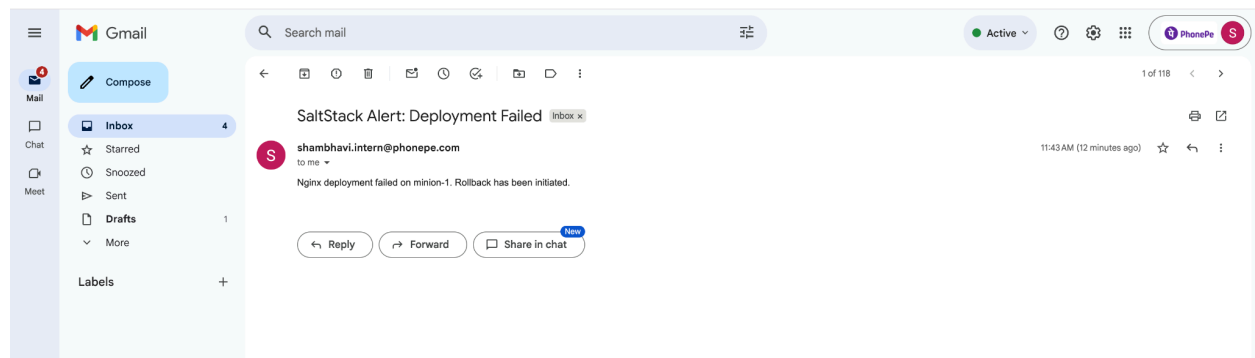
Salt's SMTP module is written in Python. It requires specific libraries to "speak" to the internet. You are installing the underlying tools that Salt needs to make an outgoing connection to Google.

ubuntu@salt-master:~\$ sudo apt install python3-requests

ubuntu@salt-master:~\$ sudo systemctl restart salt-master

ubuntu@salt-master:~\$ sudo salt-run saltutil.sync_runners

ubuntu@salt-master:~\$ sudo salt-run state.orchestrate orch.web_rolling_update



```

unauthenticated:master:/srv/salt/oron sudo salt-run state.orchestrate orcn.web_rolling_update
[ERROR ] [out]: 'highstate', 'ret': {'minion-1': {'pkg_|-nginx_packages_|-nginx_packages_|-installed': {'name': 'nginx_packages', 'changes': {}, 'result': True, 'comment': 'All specified packages are already installed', '___sls___': 'nginx.init', '___run_num___': 0, 'start_time': '11:43:12.224237', 'duration': 46.636, '___id___': 'nginx_packages'}, 'file_|-nginx_config_dir_|-etc/nginx_|-recurse': {'name': '/etc/nginx', 'changes': {}, 'result': False, 'comment': 'check_cmd determined the state failed', '___sls___': 'nginx.init', '___id___': 'nginx_config_dir', '___run_num___': 1, 'start_time': '11:43:12.272878', 'duration': 24.298, 'cmd_|-nginx_dns_resolver_|-echo "resolver ": cat /etc/resolv.conf |awk '{print $2}\' : echo "}" | tr '\n' '\ ' > /etc/nginx/resolver.conf | run' ('changes': {}, 'result': False, 'duration': 0.005, 'start_time': '11:43:12.301244', 'comment': 'One or more requisite failed: nginx.init.nginx_config_dir', '___run_num___': 2, '___sls___': 'nginx.init', '___id___': 'nginx_dns_resolver', 'name': 'echo "resolver ": cat /etc/resolv.conf |awk '{print $2}\' : echo "}" | tr '\n' '\ ' > /etc/nginx/resolver.conf'), 'service_|-nginx_service_|-nginx_|-running': ('changes': {}, 'result': False, 'duration': 0.006, 'start_time': '11:43:12.303538', 'comment': 'One or more requisite failed: nginx.init.nginx_dns_resolver, nginx.init.nginx_config_dir', '___run_num___': 3, '___sls___': 'nginx.init', '___id___': 'nginx_service', 'name': 'nginx')}}}
salt-master:master:
-----
ID: update_minion_1
Function: salt.state
Result: False
Comment: Run failed on minions: minion-1
Started: 11:43:11.216240
Duration: 1084.676 ms
Changes:
-----
minion-1:
-----
ID: nginx_packages
Function: pkg.installed
Result: True
Comment: All specified packages are already installed
Started: 11:43:12.224237
Duration: 46.636 ms
Changes:
-----
ID: nginx_config_dir
Function: file.recurse
Name: /etc/nginx
Result: False
Comment: check_cmd determined the state failed
Started: 11:43:12.272878
Duration: 24.298 ms
Changes:
-----
ID: nginx_dns_resolver
Function: cmd.run
Name: echo "resolver ": cat /etc/resolv.conf |awk '{print $2}\' : echo "}" | tr '\n' '\ ' > /etc/nginx/resolver.conf
Result: False
Comment: One or more requisite failed: nginx.init.nginx_config_dir
Started: 11:43:12.301244
Duration: 0.005 ms
Changes:
-----
ID: nginx_service
Function: service.running
Name: nginx
Result: False
Comment: One or more requisite failed: nginx.init.nginx_dns_resolver, nginx.init.nginx_config_dir
Started: 11:43:12.303538
Duration: 0.006 ms
Changes:
-----
Summary for minion-1
-----
Succeeded: 1
Failed: 3
-----
Total states run: 4
Total run time: 70.945 ms
-----
ID: rollback_minion_1
Function: salt.state
Result: True
Comment: States ran successfully. No changes made to minion-1.
Started: 11:43:12.301166
Duration: 409.253 ms
Changes:
-----
ID: alert_admin
Function: salt.function
Name: smtp.send_msg
Result: True

```

Salt Beacons

A Beacon is a small process that runs on the **Minion**. It constantly watches a specific "event." When that event occurs (e.g., a file is deleted, or a service stops), the Beacon sends an **Event** to the Salt Master's **Event Bus**.

Right now, our orchestration runs only when **you** execute the command. If we add a **Beacon** and a **Reactor**:

1. If someone manually logs into Minion-1 and breaks the Nginx config.
2. The **Beacon** detects the change.
3. The **Reactor** automatically runs your **web_rolling_update** orchestration to fix it.

Create a Master-side listener that "hears" the crash and automatically triggers the Orchestration pipeline to repair the service and roll back to a **stable.conf** if necessary.

On **minion-1**, we need to tell Salt what to watch. You can do this by creating a file in **/etc/salt/minion.d/**

Step 1.1 : sudo vim /etc/salt/minion.d/beacons.conf

None

```
beacons:
  service:
    - services:
        nginx:
            onchangeonly: True # Only fire an event when the status CHANGES
    - interval: 10           # Check every 10 seconds
```

Step 1.2 : `sudo systemctl restart salt-minion`

Step 1.3 : `sudo salt-run state.event pretty=True`

This is to check the logs of the nginx which we will run on the master

Step 1.4 : `sudo systemctl stop nginx`

Here we are stopping the nginx

```
[ubuntu@salt-master:/srv/salt/orch$ sudo salt-run state.event pretty=True
salt/beacon/minion-1/service/nginx      {
  "_stamp": "2026-01-12T09:05:49.402198",
  "id": "minion-1",
  "nginx": {
    "running": true
  },
  "service_name": "nginx"
}
salt/beacon/minion-1/service/nginx      {
  "_stamp": "2026-01-12T09:06:09.366015",
  "id": "minion-1",
  "nginx": {
    "running": false
  },
  "service_name": "nginx"
}
```

Setting up Reactor

The Reactor is **Event-based**. It happens 24/7, even when you are sleeping and no one is touching the Salt Master.

- **Scenario:** At 3:00 AM, a random disk error causes Nginx to crash, or a junior admin manually logs into the server and accidentally stops the service.

- **Action:** The **Beacon** "sees" the crash and tells the Master. The **Reactor** hears it and fixes it.
- **Goal:** To maintain "Up-time" against unplanned accidents or crashes.

we must tell the Salt Master to listen for the specific "shout" coming from the Minion's service beacon.

Step 1.1 : sudo vim /etc/salt/master.d/reactor.conf

```
None
reactor:
  # This matches the tag sent by the service beacon when Nginx status changes
  - 'salt/beacon/*/service/nginx':
    - /srv/salt/reactor/remediate_nginx.sls
```

Step 1.1 : sudo mkdir -p /srv/salt/reactor

Step 1.1 : sudo vim /srv/salt/reactor/remediate_nginx.sls

```
None
# Only trigger if the beacon reports that Nginx is NOT running
{% if data.get('nginx', {}).get('running') == False %}
trigger_safe_orchestration:
  runner.state.orchestrate:
    - args:
      - mods: orch.web_rolling_update
{% endif %}
```

Step 1.1 : sudo systemctl restart salt-master

Step 1.1 : sudo salt-run state.event pretty=True

Step 1.1 : sudo systemctl stop nginx

```
-----BEGIN PGP MESSAGE-----
salt/auth {
  "stamp": "2026-01-12T09:21:54.961503",
  "act": "accept",
  "id": "minion-01",
  "pub": "-----BEGIN PUBLIC KEY-----\nMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAmG42cRiQmZA/tz6e11vZbQDwbeifmj97kKvqnC9h5/snQBgS2WpSbj/zpeqAjPakksVtp2b4zBKmUp1sAw\nuU5GpnaP7ycz/L3x1lo5dgCZrP6pYWD0rFWgshHXe6qRcS5rD85UnV6+QentWor1070/JV1tpEC/jakcY1luM\n\nA6Mlq8XqA8o26t8omAjjV5F0piyLgFIUTLqB0Ar1PdgiunDnkwRfgo8zt36RL8t\n\n6/aTSdzN65szHrXX/WwAgf1tiniEmGOKLXnm/1nmFtZPZN4LBB1k8xFQqP65m\ntwIDAQAB\n-----END PGP MESSAGE-----"
  "result": true
}
minion_start {
  "stamp": "2026-01-12T09:21:55.517746",
  "cmd": "minion_event",
  "data": "Minion minion-01 started at Mon Jan 12 14:51:55 2026",
  "id": "minion-01",
  "pretag": null,
  "tag": "minion_start",
  "ts": 1768209715
}
salt/minion/minion-01/start {
  "stamp": "2026-01-12T09:21:55.555834",
  "cmd": "minion_event",
  "data": "Minion minion-01 started at Mon Jan 12 14:51:55 2026",
  "id": "minion-01",
  "pretag": null,
  "tag": "salt/minion/minion-01/start",
  "ts": 1768209715
}
salt/beacon/minion-1/service/nginx {
  "stamp": "2026-01-12T09:22:19.404407",
  "id": "minion-1",
  "nginx": {
    "running": false
  },
  "service_name": "nginx"
}
salt/run/20260112092219706415/new {
  "stamp": "2026-01-12T09:22:19.708107",
  "fun": "runner.state.orchestrate",
  "fun_args": {
    {
      "mods": "orch.web_rolling_update"
    }
  },
  "jid": "20260112092219706415",
  "user": "Reactor"
}
salt/run/20260112092219832977/args {
  "stamp": "2026-01-12T09:22:19.898823",
  "args": {
    "arg": [
      "nginx.init"
    ],
    "expect_minions": true,
    "kwarg": {
      "concurrent": false,
      "queue": false
    },
    "ret": "",
    "ssh": false,
  }
}
```

```

"changes": {
  "out": "highstate",
  "ret": {
    "minion-1": {
      "cmd_|-nginx_dns_resolver_|-(echo \"resolver \"; cat /etc/resolv.conf |awk '{print $2}' ; echo \";\\n\" ) | tr '\\n' ' ' > /etc/nginx/re
        __id__: "nginx_dns_resolver",
        __run_num__: 2,
        __sls__: "nginx.init",
        "changes": {},
        "comment": "One or more requisite failed: nginx.init.nginx_config_dir",
        "duration": 0.005,
        "name": "(echo \"resolver \"; cat /etc/resolv.conf |awk '{print $2}' ; echo \";\\n\" ) | tr '\\n' ' ' > /etc/nginx/resolver.conf",
        "result": false,
        "start_time": "14:52:21.602540"
      },
      "file_|-nginx_config_dir_|-/etc/nginx_|-recurse": {
        __id__: "nginx_config_dir",
        __run_num__: 1,
        __sls__: "nginx.init",
        "changes": {},
        "comment": "check_cmd determined the state failed",
        "duration": 12.443,
        "name": "/etc/nginx",
        "result": false,
        "start_time": "14:52:21.587904"
      },
      "pkg_|-nginx_packages_|-nginx_packages_|-installed": {
        __id__: "nginx_packages",
        __run_num__: 0,
        __sls__: "nginx.init",
        "changes": {},
        "comment": "All specified packages are already installed",
        "duration": 40.827,
        "name": "nginx_packages",
        "result": true,
        "start_time": "14:52:21.542101"
      },
      "service_|-nginx_service_|-nginx_|-running": {
        __id__: "nginx_service",
        __run_num__: 3,
        __sls__: "nginx.init",
        "changes": {},
        "comment": "One or more requisite failed: nginx.init.nginx_dns_resolver, nginx.init.nginx_config_dir",
        "duration": 0.005,
        "name": "nginx",
        "result": false,
        "start_time": "14:52:21.605620"
      }
    }
  },
  "comment": "Run failed on minions: minion-1",
  "duration": 1702.581,
  "name": "update_minion_1",
  "result": false,
  "start_time": "14:52:19.896920"
}
},
"out": "highstate",
"retcode": 1
},
"success": false,
"user": "Reactor"
}
salt/beacon/minion-1/service/nginx {
  _stamp: "2026-01-12T09:22:29.327817",
  id: "minion-1",
  nginx: {
    "running": true
  },
  "service_name": "nginx"
}

```

1. The Detection (The Beacon)

At timestamp 09:22:19, the Beacon on minion-1 sent a message to the Master's event bus. Tag: salt/beacon/minion-1/service/nginx Data: "running": false This is the "shout" for help. Nginx was stopped, and the Beacon caught it instantly.

2. The Decision (The Reactor)

Milliseconds later, the **Reactor** saw that specific tag and consulted your mapping.**User:**

Reactor Function: runner.state.orchestrate **Args:** mods:

orch.web_rolling_update The Reactor didn't just try to restart Nginx; it triggered your professional **Orchestration** pipeline to ensure a safe, validated recovery.

3. The Attempted Update (The Shield)

The Orchestration started the `update_minion_1` job. However, because you still had the `80xxx` port error in your config, the **Shield** blocked it: **ID:** `nginx_config_dir` **Comment:** `check_cmd determined the state failed` **Result:** `false` This is exactly what we want. The system refused to apply the "poison" config even during an automated recovery.

4. The Self-Healing Loop (Rollback & Alert)

Because the update failed, the Orchestration triggered the `onfail` steps:

- **Step A: Rollback**
ID: `rollback_minion_1` triggers `nginx.previous_stable` **Action:** It successfully restarted Nginx using the stable configuration.
- **Step B: Alert**
Function: `smtp.send_msg` **Result:** `true` The Master successfully sent the email alert to your inbox at `09:22:26`.

5. The Resolution (Success)

Finally, at timestamp `09:22:29`, the Beacon fired one last time:

nginx: `{"running": true}` **The loop is complete.** The system detected a failure, tried to update, hit a config error, rolled back to a safe version, alerted you, and confirmed the service is back online.